

Tool to Predict Grid and Block Sizes

Graphics Processing Units (GPUs): Architecture and Programming

Alex Lee, Arman Keshavarz

Abstract

Choosing the appropriate thread and block configuration is one of the most important factors influencing GPU kernel performance because poorly selected dimensions can limit occupancy, underutilize memory bandwidth or create bottlenecks leading to suboptimal performance despite the same computation. However, determining the optimal configuration depends on many things: kernel characteristics, device architecture, and problem size making manual tuning time consuming and error prone.

In this project, we design a system that analyzes kernel characteristics, GPU make, and problem size to predict efficient thread-block configurations automatically. Our approach draws on the mixture-of-experts idea to split the training model into multiple key target kernels for which we train.

Our results show that the system can learn meaningful relationships between problem size, kernel type, and GPU configuration, outperforming arbitrary parameter choices, but accuracy is limited by gaps and artifacts in the data generation process. With improved sampling of thread/block combinations and better coverage of valid configurations, the predictive model can become more reliable.

Introduction

Because GPUs rely on massive parallelism and hardware-dependent scheduling, even small changes in launch configuration can dramatically affect occupancy, warp efficiency, and overall throughput. The thread and block parameters directly affect how many threads run concurrently, how efficiently warps are scheduled, and how well memory bandwidth is utilized. In the real world, kernels can be massively slower when launched with suboptimal parameters even when their code is perfectly written and as GPUs grow increasingly complex, the tuning becomes more important and more difficult. For example, in scientific computing, robotics, machine learning and large-scale simulations, kernels are launched millions of times during training or inference and poorly tuned configurations don't just waste a

few milliseconds but they compound into hours of lost computation time and energy. Despite decades of GPU development there is still no widely adopted method that reliably predicts efficient launch parameters for arbitrary kernels.

Our project addresses this challenge by learning from representative kernel behaviors rather than analyzing every new kernel as a completely unknown problem. We construct a diverse set of preset kernels that cover compute-bound, memory-bound, mixed, and irregular patterns and sweep across many configurations to gather data. This collection forms a controlled setting where each kernel's characteristics and optimal configurations are known. Once identified, this knowledge is used in our predictive model. When presented with an unseen kernel, we extract its features, match it to the closest preset kernel behaviors and use different models to predict good thread/block combinations without requiring extensive testing. This system allows us to generalize across kernel types while leveraging data which allows fast configuration selection with minimal tuning.

Literature Survey

Optimizing thread and block configurations for kernels is closely connected to other research areas in performance modeling and architecture-aware optimization and while no major body of work directly solves this exact problem, a large set of methods deals with similar challenges. We can organize literature into 3 major categories, analytical performance models, online search, and ML based kernel modeling.

Some analytical models include the Roofline model, occupancy calculators provided by NVIDIA, or studies of throughput modeling. The Roofline model is a simple framework that predicts the maximum possible performance of a kernel based on FLOPs per byte of data, hardware ceilings. The occupancy calculator estimates how many warps can run concurrently on an SM given threads/block, registers per thread, shared memory usage and hardware limits. Given a block size, they compute achieved occupancy and whether increasing or decreasing block sizes raises occupancy. Throughput modeling might include

papers such as “Demystifying GPU Microarchitecture Throughput” which builds tables of latency, pipelining, etc. and gives what a kernel is bottlenecked by. Some strengths are that they are hardware-centered and fast, but they are too coarse to predict the exact optimal block size for real kernels. Additionally, they cannot fully capture irregular memory access, divergence, or complex instructions. These can tell why a kernel might perform well or poorly, but they cannot reliably predict which thread-block configuration is best which is why a more data-driven approach is needed.

Auto-tuning and search-based methods evaluate many possible configurations to find the best one. Major frameworks include Kernel Tuner for OpenCL/CUDA and ATLAS. ATLAS is originally designed for CPU-side operations and predates GPU tuning work but introduced many concepts that were later adopted. It automatically generates many variants of low level kernels and searches over many factors to select the fastest version and use it at runtime. Kernel Tuner is a Python tool that auto-tunes CUDA kernels by sweeping different launch configurations and parameters so for user-supplied kernels it can vary threads/block, number of blocks, tile/block shape etc... and runs each variant and times the execution. While it is simple to use and can apply intelligent search strategies, it requires running the kernel many times which is extremely slow and does not generalize to new kernels where tuning must be repeated. Therefore, this is not suitable for real-time or large-scale production workloads. It would be easier to avoid the brute-force cost by learning from offline data and generalizing to new kernels.

Machine learning has increasingly been used to predict GPU kernel performance, power consumption and resource usage since they can learn complex relationships between kernel code and characteristics. Examples of representative studies may be like “Predicting GPU Kernel Execution Time using Static Analysis” or “Using Machine learning to Predict GPU Kernel Runtime” or Ansor. In the first two, they use instruction counts and pipeline models with ML regression and other characteristic usage to predict runtime but they predict runtime not optimal launch configurations. Ansor uses hierarchical search and learned cost models to prune configurations which can predict promising candidates. Fundamentally, they still depend on online execution, cannot generalize fully, and require large training

datasets. Due to that, they only output for domain-specific workloads while our goal is to eliminate online searches and predict for arbitrary user-written kernels.

Proposed Idea

A central part of our system is the kernel type decider, a module that automatically analyzes an arbitrary CUDA kernel and assigns it to one of a set of representative kernel classes. Selecting thread and block sizes depends heavily on the type of workload that a kernel represents so compute-bound kernels prefer different configuration than memory-bound kernels etc.... To address this gap, we designed a static feature-based kernel classifier that analyzes the kernel's PTX and high level structure, extracts meaningful behavioral indicators and assigns the kernel the closest class among our present kernels. The kernel decider allows us to turn the large space of all CUDA kernels into representative behaviors and this makes it possible to generalize across similar execution patterns, select thread/block configurations that are known to work well for that class and avoid online auto-tuning/brute-force sweeping.

The kernel classification system operates in 3 stages, compiling CUDA to PTX, extracting structural and instruction features, and routing the kernel to a class using rule-based reasoning. PTX is used because it is architecture-independent but shows low-level behavior. It shows precise instructions like loads, stores, atomics and includes control-flow, loop structure and shared memory usage. These are all features directly related to performance. The feature extraction uses regular expressions and instruction counters to count loads, stores, compute, int-ops, bit_ops and other things like branches, nested_loops, etc.... These counts help determine whether a kernel is memory bound or compute heavy or bitmask oriented and control flow metrics indicate graph workloads or stencil patterns. This feature set is chosen because they directly correlate with the preset kernel classes we benchmarked, for example N-body has heavy floating-point compute and nested loops, while BFS has branches, atomics, and indirect access, while scatter/gather has indirect memory and loads/stores. We chose a rule-based classifier because GPU kernels fall into natural categories (matmul, reduction, BFS) and we don't have

thousands of labeled real-world kernels to use to train. So, rule-based routing is the most robust solution for the first stage.

To train our predictive model for selecting thread and block configurations, we require a large dataset of kernel executions measured under a wide variety of launch parameters and input sizes. We start with random sizes for our problem size input which prevents the model from overfitting to a structured sequence of input sizes and ensures coverage across many different data footprints. We only choose sizes of problem sizes to generate that would be appropriate for GPU modeling. For example, inputs from 100k to 10 million which target realistic application workloads and effectively parallelizable problem sizes. For small inputs like $N < 100k$, kernel launch overhead dominates execution time so for small sizes, all configurations look the same. We sweep through threads per block from 16 to 1024 which are practical choices because we start from a meaningful minimum amount of parallelism to common choices for memory and compute heavy kernels (64, 128, 256) to the maximum legal dimensions on most GPUs. So, we capture both under-occupied and fully-occupied modes. We also sweep the block counts which range from tiny grids up to the maximum grid dimension for 1D launches (in the 2D we also do so). This enables exploration of warp pressure, launch overhead scaling and memory bandwidth saturation. We repeat each configuration multiple times because GPU execution time is noisy due to clock frequency fluctuations and other things. Repeating each configuration reduces variances and ensures the training data is stable. We log the important information which includes problem size, threads per block, number of blocks, time in milliseconds, which GPU it was run on, and which kernel it corresponds to.

Once we classify each kernel into a meaningful category, the next step is to predict the optimal CUDA configuration for unseen kernels. To do this, we train several machine learning models capable of learning the relationship between kernel features, device-dependent launch parameters and measured runtime. For our model, given a classified kernel, problem size, and GPU type, we predict threads per block and number of blocks. We train four different models: gradient boosted trees, linear regression, random forest regression and a neural network. Using multiple models captures different relationships;

for example, XGBoost handles nonlinear interactions well, linear regression is fast and interpretable, random forest is good at capturing discrete jumps and robust to noise, and neural networks can learn smooth nonlinear mappings. This allows us to compare performance, check consistency and potentially compare later. We split the dataset into an 85% training set and 15% validation set using random splits to broadly cover different kernel types, GPU models, and problem sizes. Because threads per block and number of blocks vary continuously across problem sizes, we treat them as regression targets, allowing the model to interpolate and generalize to unseen configurations. This machine learning subsystem therefore maps kernel class, problem size, and GPU model into launch configurations without requiring online tuning. From this, we tested the viability of the models and after adding features of N_{bin} , $N_{squared}$, N_{log} and N_{sqrt} , random forest turned out the best so this is what we primarily used.

Experimental Setup

Experiments were performed on 2 machines with the following specifications:

GPU: NVIDIA GeForce RTX 2080 Ti, CPU: Intel Xeon E5-2660 v3, RAM: 256 GB,
Storage: 1 TB SSD, OS: Linux 5.14.0-570.55.1.el9_6.x86_64

GPU: NVIDIA TITAN V, CPU: Intel(R) Xeon(R) Gold 5118 CPU, RAM: 128 GB,
Storage: 372 GB SSD, OS: Linux 5.14.0-570.55.1.el9_6.x86_64

The following software stack was used:

Cuda toolkit: Cuda 12.2, Nvidia driver version: 580.76.05, GCC: 11.5.0, Python: 3.9.23

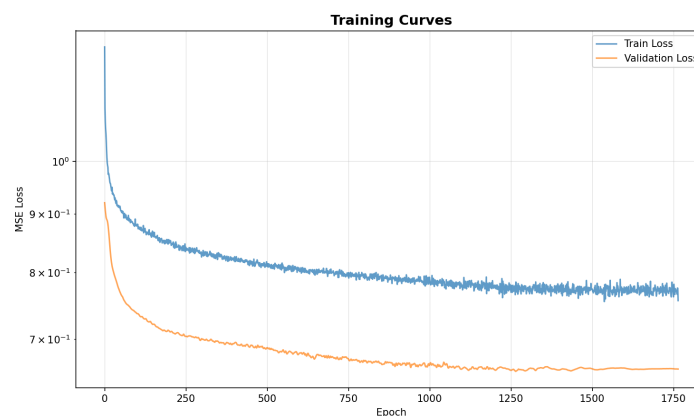
We benchmarked a set of preset kernels designed to cover diverse GPU workload patterns which include: copy, scatter/gather, strided memory access, SAXPY, prefix sum, reduction, matrix multiply, convolution, stencil, bitmask, N-body, dynamic parallelism and graph BFS. Each kernel has one .cu file

containing its implementation with a driver that sweeps configuration parameters and a csv output file storing results.

For each kernel we generated performance measurements using a standardized procedure where we select 1000 input sizes uniformly selected from a tested range of reasonable elements for each of the kernels. Then we sweep from 16 to the maximum number of 1 dimensional units for threads and blocks which covers the entire practical configuration space. Each run was timed using CUDA and a csv with the following schema was produced containing N, threads_per_block, num_blocks, time_ms, type, GPU across all kernels.

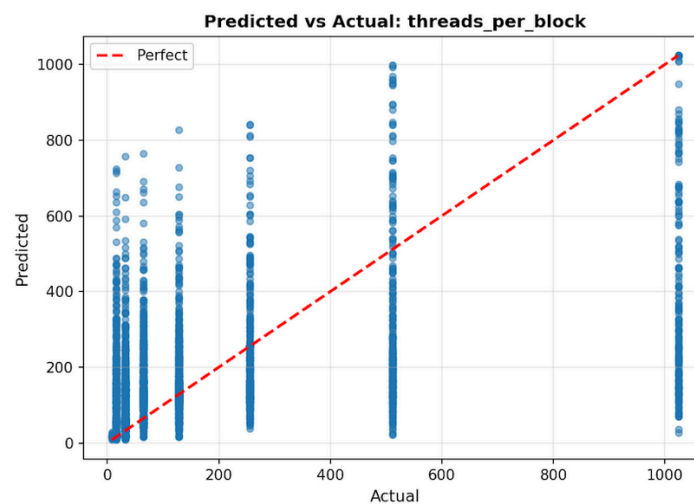
Experiments & Analysis

The goal of this subsystem is to predict CUDA launch parameters solely from problem size, kernel class, and GPU model without running the kernel online. Success requires accurate regression predictions for both the parameters, generalization to unseen N values, stable training behavior, and results that are accurate (predicted vs actual plots that cluster close to the diagonal).

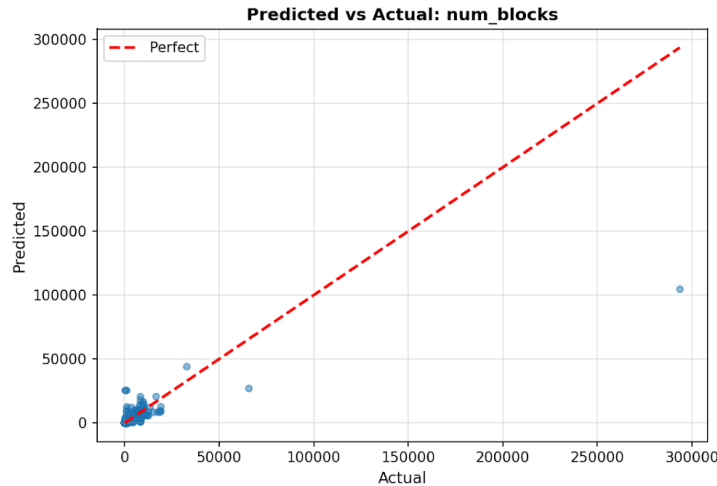


The training curves for training the small neural network we did show rapid loss drop in the first 50 to 100 epochs and validation loss is consistently lower than training loss which is not ideal for random neural nets showing evidence of overfitting to the training set. For this reason we decide to move on to the

random forest regressor we the the plot above for. In this random forest regressor we did some feature engineering with the N value which corresponds to problem size and did $\sqrt{N}$, N^2 , and $\log(N)$ as new features for the random forest algorithm and this significantly helped reduce our MAE as we ended up with a MAE of 120.6330 for threads per block and an MAE of 345.1642 for number of blocks. These values seem extremely high at first but as we will see when we did actual testing predictions for specific kernels that the Random Forest regressor actually did a pretty good job predicting the optimal configuration given the problem size, GPU type and kernel.



In this scatter plot for the random forest regressor, the `threads_per_block` shows clustering in the actual values, this is because the data only has a few legal block sizes of 16, 32, 64, 128, 256, 512, 1024 which appear as the vertical columns. Because random forest is a continuous regressor and because actual values jump between multiples, predictions fall between the legal block sizes. Therefore the model does not perfectly match the discrete buckets and this is expected because the regressor averages across samples instead of snapping to the hard categories. We see that the model successfully captures relative trends (larger inputs require larger block sizes) but exact matching is limited due to the sparsity of data available for unusual block sizes. This confirms that post-processing rounding is essential to map predictions back to legal CUDA values which is why in the final estimate, we snap the threads to multiples of 32 because NVIDIA GPUs schedule threads in warps of 32.



The number of block distributions exhibits different behaviors where most actual values cluster near small block counts of 0 to 20k while a few kernels are outliers and require massive parallelism. Outliers are difficult to predict and extreme configurations may require explicit rule-based adjustments because random forests struggle with large extrapolations and the sparse cases do not dominate the regression surface. On the other hand for the large majority of the data, predictions remain near the dense region and we can see outputs are relatively good because most of the points cluster near the perfect line at small values (general correlation is captured). Because `num_blocks` aren't as discrete binned as the `threads_per_block`, the values are continuous and many similar examples exist so in the dense region most points line up close to the diagonal and we can say this is more reliable.

Dynamic Parallelism Kernel Accuracy

	Accuracy Raw	Accuracy Percentage
Thread Matches	407/848	48.0%
Block Matches	582/848	68.6%

Here we test accuracy to see how well our system works on a given kernel. To test, we run our inference on a dynamic parallelism kernel and obtain a thread and block matches output. The thread value is matched to see if it was in our original dataset using a best of 3 filter. In other words, it would be

counted as accurate if the thread value we generated was the thread per block value in the top 3 configurations for that problem size. A similar experiment was done for block matches where the top 3 configurations were selected and values would be accurate if they were within 20% of the optimal value.

Looking at the performance, they seemed to be mediocre and while they are better than arbitrarily choosing a thread and block value, there is room for improvement. The accuracy for both thread and block matches were low which can be attributed to choices in the data generation that could have been changed. For example, our model was trained on data that was generated from powers of 2 since those are common values for threads per block (e.g. 16, 32, 64...). This is incompatible with our ML model which utilizes interpolation as this left a lot of holes in our data. Because of this, we would have to round the output to one of those bins which may have led to loss of accuracy. In a future work, it would be better to generate data using all multiples of 32 up to the thread limit as this would allow for better estimation without having a large gap between 2 values like 512 and 1024. The block matches seemed to do better because the values did not have to be binned but the accuracy is still only slightly higher. Because we needed to generate a lot of data, the times for each iteration could not be too long and this left us with some points that may have skewed the results. For example, having an insufficient problem size tested with the block limit in 1D would have a similar time to run as one with a smaller block value. When training the data, a large block would then resemble those of a much smaller and reasonable configuration. As a result, the timing data for extreme block counts does not reflect true performance differences but instead collapses onto a narrow band of nearly identical runtimes that skews our data. This makes it hard to learn meaningful relationships since two radically different configurations can yield nearly identical timings and interpolating can distort the block count.

Conclusions

We find that predicting kernel launch parameters is feasible but accuracy is fundamentally limited by dataset design rather than model capacity; the regression learned meaningful trends especially for

continuous values but discrete constraints prevented accurate predictions. Model performance strongly depends on how data is generated and samples since sweeping invalid configurations would make interpolation difficult and introduce large error regions. However, a data-driven, classification and regression approach shows promise as despite moderate accuracy, this showed we could meaningfully narrow the search space for thread/block sizes and outperform naive selection strategies.

References

Brandt, Alexander, Marc Moreno, Maza, Taabish, Jeshani, Linxiao, Wang, Davood, Mohajerani, Jeeva, Paudel. "Dynamically Finding Optimal Kernel Launch Parameters for CUDA Programs." *Proceedings of the 33rd Annual International Conference on Computer Science and Software Engineering*. IBM Corp., 2023.

G. Alavani, K. Varma and S. Sarkar, "Predicting Execution Time of CUDA Kernel Using Static Analysis," 2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCLOUD/SocialCom/SustainCom), Melbourne, VIC, Australia, 2018, pp. 948-955, doi: 10.1109/BDCLOUD.2018.00139. keywords: {Graphics processing units;Kernel;Delays;Instruction sets;Hardware;Computational modeling;Predictive models;High Performance Computing, GPGPU, Prediction Technique, Analytical Model},

H. Wong, M. -M. Papadopoulou, M. Sadooghi-Alvandi and A. Moshovos, "Demystifying GPU microarchitecture through microbenchmarking," 2010 IEEE International Symposium on Performance Analysis of Systems & Software (ISPASS), White Plains, NY, USA, 2010, pp. 235-246, doi: 10.1109/ISPASS.2010.5452013. keywords: {Microarchitecture;Graphics processing unit;Samarium;Computer architecture;Delay;Clocks;Kernel;Registers;Performance analysis;Hardware},

Lianmin Zheng, , Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, Ion Stoica. "Ansor: Generating High-Performance Tensor Programs for Deep Learning." *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020.

Marcos Amaris, Raphael Camargo, Daniel Cordeiro, Alfredo Goldman, Denis Trystram, Evaluating execution time predictions on GPU kernels using an analytical model and machine learning techniques,

Journal of Parallel and Distributed Computing, Volume 171, 2023, Pages 66-78, ISSN 0743-7315,
<https://doi.org/10.1016/j.jpdc.2022.09.002>.

Nersc. “Roofline Performance Model¶.” *Roofline Performance Model - NERSC Documentation*, NERSC,
2020, docs.nersc.gov/tools/performance/roofline/.

Schoonhoven, Richard, et al. “Going Green: Optimizing GPUs for Energy Efficiency through
Model-Steered Auto-Tuning.” arXiv:2211.07260, arXiv, 14 Nov. 2022. *arXiv.org*,
<https://doi.org/10.48550/arXiv.2211.07260>.